

Programação Orientada a Objetos - CST em Análise e Desenvolvimento de Sistemas

Classe abstrata, interface e polimorfismo

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Classe abstrata

Jogo Java of Empires

Fortemente inspirado no *Age of Empires* da Microsoft – <https://www.ageofempires.com>



Fonte: Microsoft

- **Aldeão**
- Vida: 25
- Ataque: 1
- Velocidade: 0.8



Fonte: Microsoft

- **Arqueiro**
- Vida: 35
- Ataque: 2
- Velocidade: 1

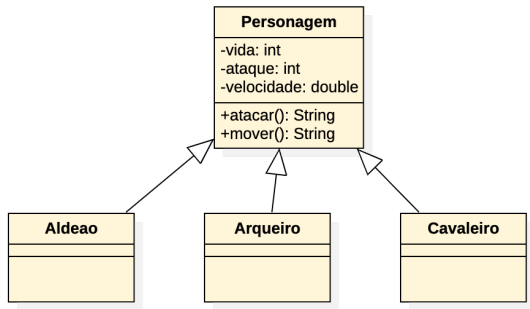


Fonte: Microsoft

- **Cavaleiro**
- Vida: 50
- Ataque: 3
- Velocidade: 2

Jogo Java of Empires

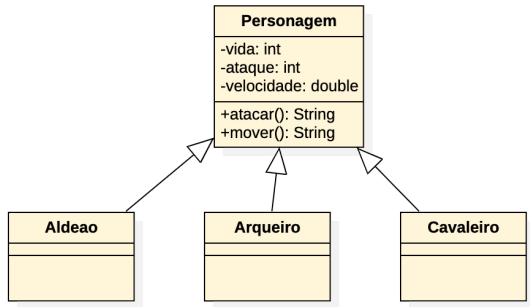
Proposta de modelagem 1



- Se o método mover está em Personagem, todas as subclasses têm esse método
- Não são obrigadas a sobrescrever (herdam o mesmo comportamento)
- Isso pode ser um problema: todos se movem igual
- Pergunta: como garantir que cada personagem tenha seu próprio jeito de mover?

Jogo Java of Empires

Proposta de modelagem 1



- No jogo, só jogamos com Aldeao, Arqueiro e Cavaleiro
- Mas a modelagem permite criar um Personagem genérico
- Isso não faz sentido para o jogo
- Solução: tornar Personagem abstrata

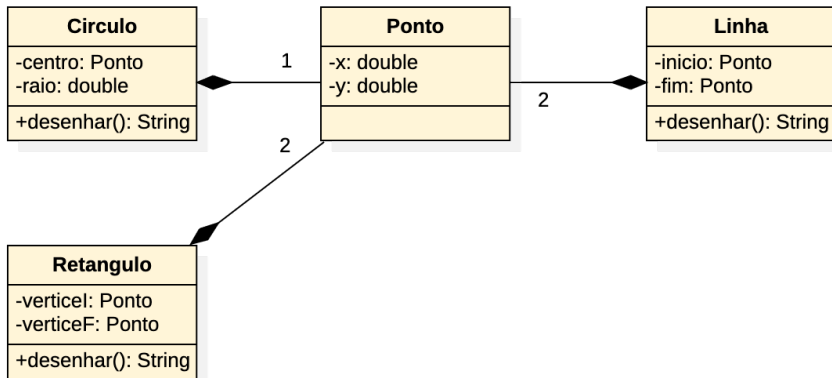
```
Personagem p = new Personagem(); // Não deveria ser
                                permitido
p.mover();
```

Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo

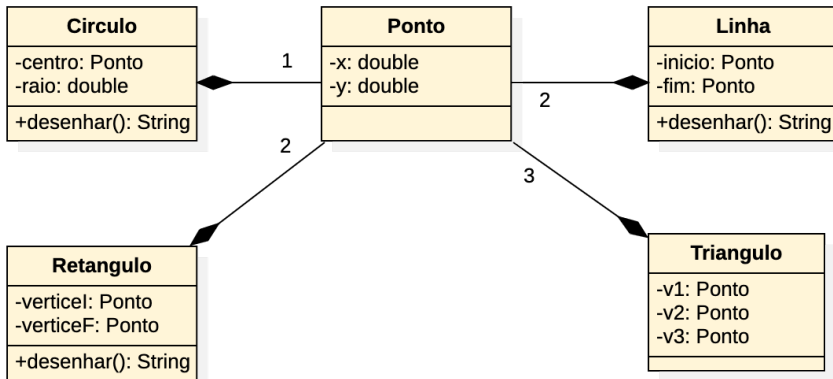
Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo



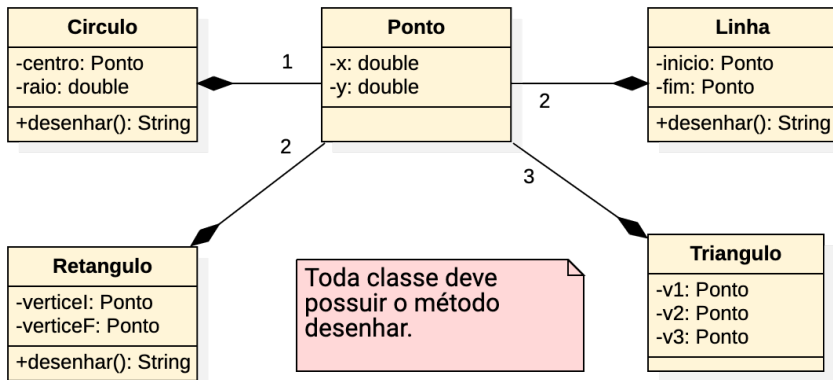
Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo
- Triângulo



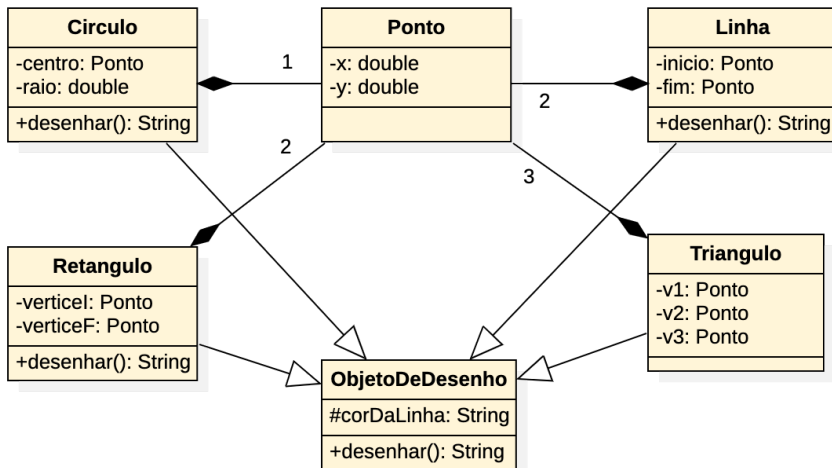
Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo
- Triângulo



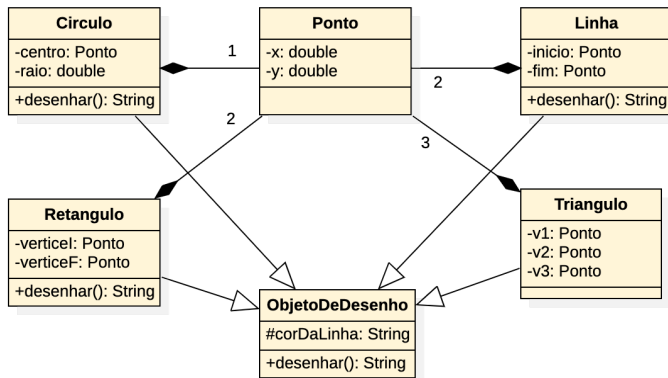
Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo
- Triângulo



Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo
- Triângulo



Toda classe deve possuir o método desenhar.
Esse modelo ainda não garante isso!

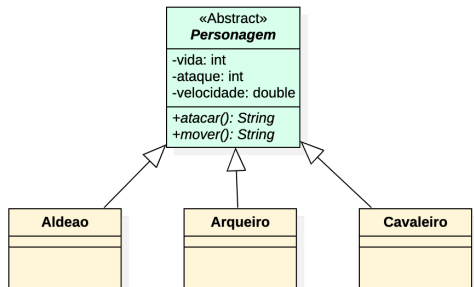
Classe abstrata em Java

- Pode ter construtores, atributos e métodos, mas não pode ser instanciada
 - Construtores usados para inicializar os atributos da classe abstrata quando uma subclasse é instanciada
 - Atributos podem ser usados para armazenar informações comuns a todas as subclasses
 - Métodos podem ser concretos (com implementação) ou abstratos (sem implementação)
- Subclasse deve sobrescrever os métodos abstratos ou também deve ser abstrata
- Métodos concretos podem ser sobrescritos, mas não é obrigatório

Use classe abstrata para criar uma base comum e obrigar subclasses a implementar métodos específicos, mas permita que compartilhem código comum

Java of Empires

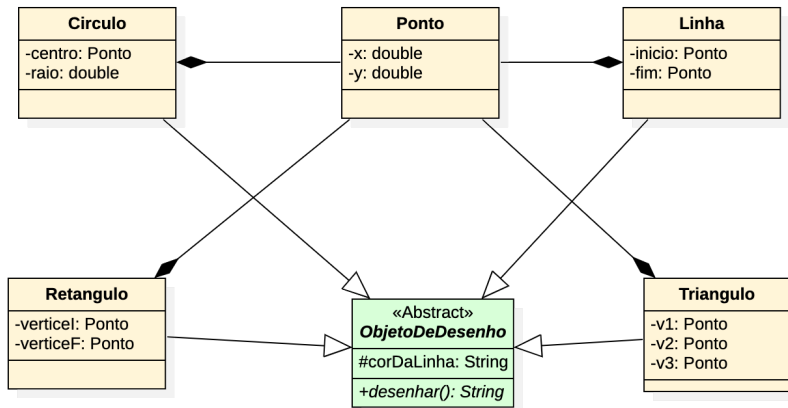
Proposta de modelagem 2



- Personagem é uma **classe abstrata**
 - Não pode ser instanciada
- Os métodos mover e atacar são **abstratos**
 - Obriga que as subclasses sobrescrevam esses métodos
- Cada objeto das classes concretas Aldeao, Arqueiro e Cavaleiro é um Personagem, e portanto, pode ser tratado como tal

```
Aldeao a = new Aldeao();
a.mover();
a.setVida(25);
```

Aplicativo para desenho vetorial 2D (geometria plana)



Esse modelo obriga, a toda classe que herdar de ObjetoDeDesenho, implementar o método abstrato desenhar()

Aplicativo para desenho vetorial 2D (geometria plana)

```
public abstract class ObjetoDeDesenho{  
    public abstract String desenhar();  
}
```

```
public class Circulo extends ObjetoDeDesenho{  
    private Ponto centro;  
    private double raio;  
  
    // A classe Circulo é obrigada a sobrescrever o método desenhar  
    @Override  
    public String desenhar(){  
        return String.format("Desenhando circulo com raio %.2f", raio);  
    }  
}
```

Interface

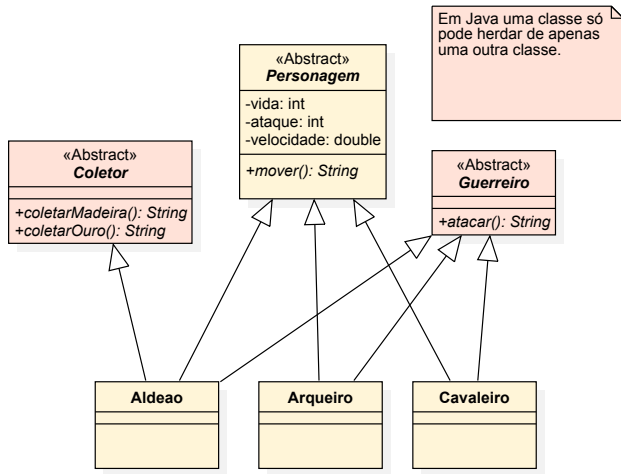
Herança múltipla

Em Java, interfaces permitem herança múltipla de tipos

- **Herança múltipla** é um conceito de orientação a objetos onde uma classe pode herdar atributos e métodos de mais de uma classe
 - A linguagem C++ possui o conceito de **herança múltipla**
- Em Java, uma classe pode herdar de apenas uma outra classe
 - Evita as complicações relacionadas a **herança múltipla de estados** – habilidade de herdar atributos de múltiplas classes
- Em Java, uma classe pode implementar várias **interfaces**, permitindo assim o conceito de **herança múltipla de tipos**
 - **herança múltipla de implementação** – habilidade de herdar as definições de métodos de múltiplas interfaces

Java of Empires

Modelagem com herança múltipla



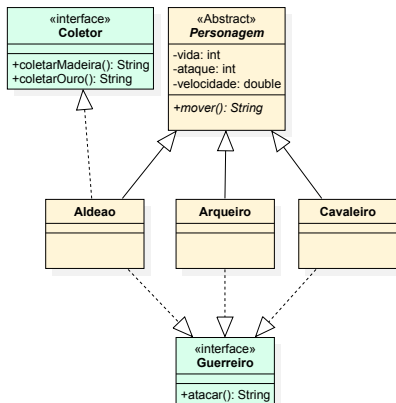
Interface

Interface: define um contrato que as classes devem seguir

- Todos os atributos são `public static final` (constantes)
- Não pertencem às classes, mas à interface
- Todos os métodos são `public abstract` por padrão
- Métodos `static` e `default` podem ter código
- Uma classe pode implementar várias interfaces

Java of Empires

Modelagem com interface



```
public interface Guerreiro{
    String atacar();
}
```

```
public class Aldeao extends Personagem implements Guerreiro, Coletor {

    @Override
    public String atacar(){
        return "Aldeao atacando";
    }

    @Override
    public String coletarMadeira(){
        return "Aldeao coletando madeira";
    }

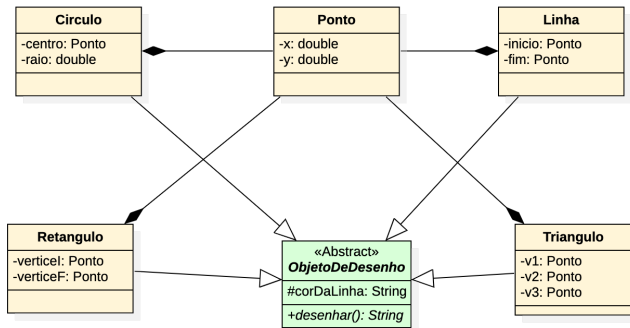
    @Override
    public String coletarOuro(){
        return "Aldeao coletando ouro";
    }

}
```

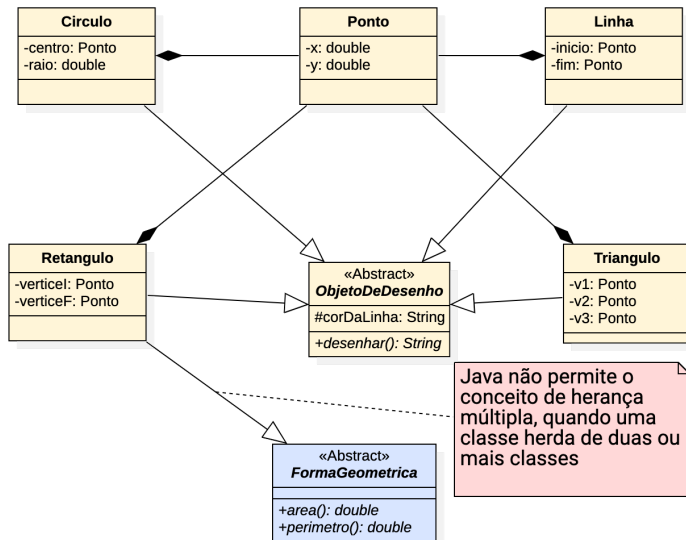
Aplicativo para desenho vetorial 2D (geometria plana)

Quais ajustes fazer para que todas as formas geométricas possam desenhar, calcular área e perímetro?

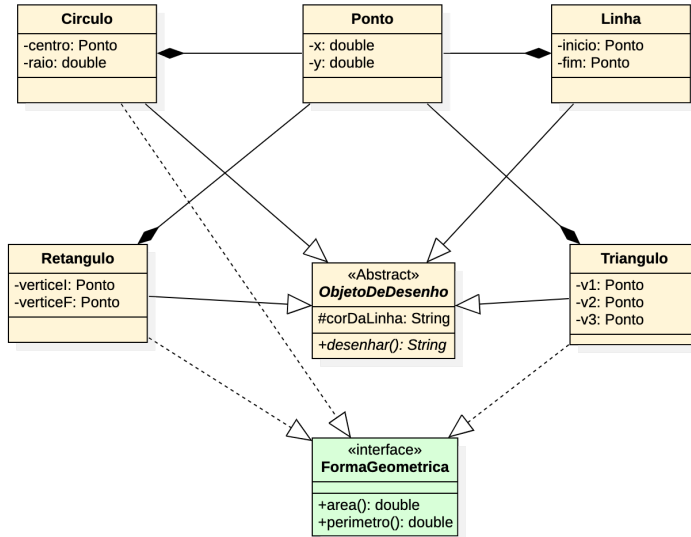
- Adicionar métodos `calcularArea` e `calcularPerimetro` em `ObjetoDeDesenho`?
- Criar uma nova classe abstrata `FormaGeometrica`?
- Usar interface para garantir os métodos?



Aplicativo para desenho vetorial 2D (geometria plana)



Aplicativo para desenho vetorial 2D (geometria plana)

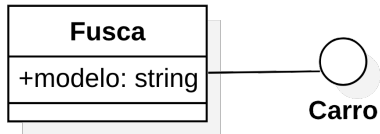
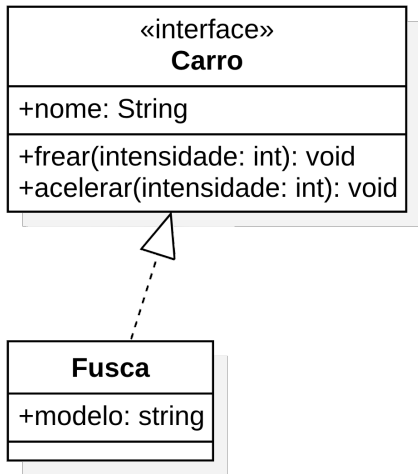


Exercício

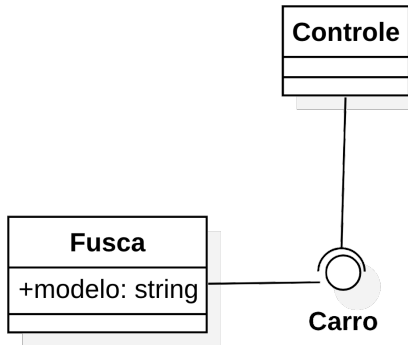
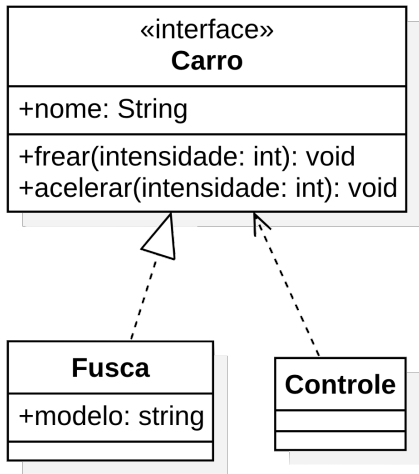
Jogo de corrida

- Um fabricante de jogo de corrida gostaria de **permitir que seu jogo fosse estendido por outras pessoas** de forma que **possam criar seus próprios carros**
- **Deve-se garantir que todos os carros possuam os mesmos métodos**
 - acelerar, frear, ligar e desligar
- Cada carro pode ter um comportamento diferente ao acelerar, frear, ligar e desligar
- Cada carro pode ser de uma marca diferente, ter uma cor diferente, um modelo diferente

Diferentes formas para representar interfaces na UML



Diferentes formas para representar interfaces na UML



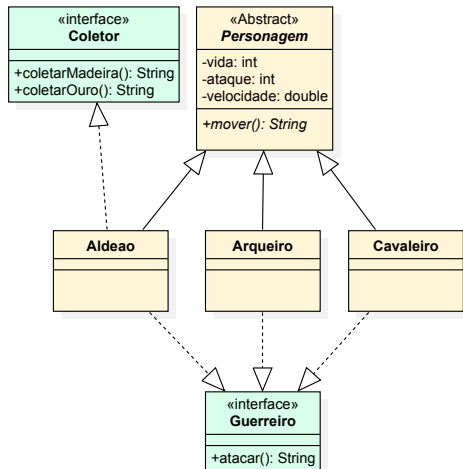
Polimorfismo

Polimorfismo

O que é polimorfismo?

- Um objeto pode ser tratado como seu tipo específico ou como um tipo mais genérico
 - Exemplo: um Aldeao pode ser usado como Personagem
- Facilita a extensão do sistema, pois novas classes podem ser adicionadas sem mudar o código geral
 - Programe para o mais genérico, não para o mais específico

Java of Empires



- Tudo que é possível fazer com um **Personagem** é possível fazer com um **Aldeao**, **Arqueiro** e **Cavaleiro**, mas não o contrário

```
Aldeao a = new Aldeao();
Personagem p = a;

a.mover(); // OK
p.mover(); // OK

a.coletarMadeira(); // OK
p.coletarMadeira(); // ERRO
```

Java of Empires

- O exército pode ter até 300 personagens
- É desejado mover todos os personagens do exército

```
Aldeao    vetA[] = new Aldeao[100];
Arqueiro  vetB[] = new Arqueiro[100];
Cavaleiro vetC[] = new Cavaleiro[100];

//omitindo a criação dos objetos

// invocando o método mover de cada objeto
for(int i = 0; i < 100; i++){
    vetA[i].mover();
    vetB[i].mover();
    vetC[i].mover();
}
```

- Essa abordagem não é recomendada, pois limita em 100 o número de personagens de cada tipo
- Se um novo tipo de personagem for criado, será necessário modificar o código dentro do laço de repetição

Java of Empires

- Com o **polimorfismo** é possível incluir novos personagens no jogo sem que seja preciso modificar boa parte do código
 - Basta adicionar um novo tipo de personagem e incluí-lo no vetor de personagens
 - O método mover será chamado de acordo com o tipo de personagem
 - Programe para o mais genérico e não para o mais específico

```
// O vetor da superclasse pode armazenar objetos das suas subclasses
Personagem vetP[] = new Personagem[300];

vetP[0] = new Aldeao();
vetP[1] = new Arqueiro();
vetP[2] = new Cavaleiro();
vetP[3] = new Soldado();

for(int i=0; i < 4; i++){
    // Polimorfismo. Em tempo de execução é determinado qual implementação será invocada
    vetP[i].mover();
}
```

Exercícios

Herança múltipla com Interfaces

- Faça um diagrama UML com classes e interfaces que permitam representar animais em um jogo
 - Arara, Avestruz, Baleia, Cachorro, Gato, Leão, Lobo, Macaco, Morcego, Pinguim, Ornitorrinco
- Todo animal possui um nome e no mínimo 2 métodos de habilidade, ou seja, ações que o animal pode realizar
 - Exemplo: cachorro late e corre, arara voa e fala, leão ruge e corre
 - Método toString deve retornar o nome do animal e de qual classe ele é. Exemplo: "Totó é um cachorro"
- Organize as classes e interfaces em uma hierarquia de forma que seja possível criar um vetor de animais e chamar suas habilidades usando polimorfismo
- Crie uma classe App para testar a implementação

Gerar folha de pagamento de uma empresa

- Existem 4 tipos de carreira na empresa:
 - **Mensalista fixo:** recebe um valor fixo por mês, independentemente do número de horas trabalhadas.
 - **Horista:** recebe um valor por hora trabalhada e um valor adicional para cada hora extra trabalhada além das 40 horas semanais. O valor da hora extra é acordado individualmente com cada funcionário.
 - **Comissionado:** recebe um salário calculado apenas sobre um percentual das vendas realizadas. O percentual das vendas é acordado individualmente com cada funcionário.
 - **Comissionado efetivo:** recebe um valor fixo por mês, independentemente do número de horas trabalhadas, mais um percentual das vendas realizadas. O percentual das vendas é acordado individualmente com cada funcionário.

Gerar folha de pagamento de uma empresa

- Modele as classes para representar os funcionários dessa empresa.
- Implemente uma rotina que gere a folha de pagamento da empresa.
 - A folha de pagamento deve conter o nome do funcionário e o valor a ser pago.
- Implemente uma rotina que aumente em 10% o salário base de todos os funcionários da carreira **comissionado efetivo**.